

Zahlen & Operatoren

<code>x / y</code>	echte Division, immer float
<code>x // y</code>	ganzzahlig, rundet ab
<code>-7 // 3</code>	-3 — nicht -2 wie in C
<code>8.0 // 2</code>	4.0 — mit float wird es float
<code>x % y</code>	Rest
<code>x ** y</code>	Potenz
<code>2 ** 100</code>	int wird beliebig groß
<code>x += 1</code>	Kurzform; kein ++
<code>x == y</code>	auch != < <= > >=
<code>int("42")</code>	auch float("0.5"), str(7)

f-Strings

`f"{score:.2f}"` — in `{ }` Variablen oder Ausdrücke, hinter `:` die Formatangabe.

Angabe	Beispiel	Ergebnis
<code>:.2f</code>	<code>f"{0.83124:.2f}"</code>	0.83
<code>:.0f</code>	<code>f"{0.83124 * 100:.0f}"</code>	83
<code>:,</code>	<code>f"{1234567:,}"</code>	1,234,567
<code>>8 / <:8</code>	<code>f"{42:>8}"</code>	Breite 8, rechts-/linksbündig
<code>:08.3f</code>	<code>f"{3.14159:08.3f}"</code>	0003.142

Verzweigungen

<code>if score >= 0.8:</code>	Doppelpunkt eröffnet den Block
<code>label = "stark"</code>	vier Leerzeichen Einrückung
<code>elif score >= 0.5:</code>	erster Treffer gewinnt
<code>label = "ok"</code>	
<code>else:</code>	
<code>label = "schwach"</code>	
<code>0.5 <= score < 0.8</code>	verkettete Vergleiche
<code>a and b, a or b, not a</code>	statt <code>&&</code> !
<code>if xs:</code>	leer zählt als falsch (NumPy: anders)

Schleifen

<code>for s in scores:</code>	direkt über die Elemente
<code>for i in range(n):</code>	0 bis ausschließlich n
<code>for i in range(2, 5):</code>	2, 3, 4
<code>for i, s in enumerate(xs):</code>	Position und Wert
<code>while bedingung:</code>	solange sie stimmt
<code>break</code>	bricht vorzeitig ab
<code>for i in range(len(xs) - 1):</code>	Inderschleife: <code>xs[i + 1]</code>

Einrückung ist Logik: Sie allein entscheidet, was wiederholt wird.

Funktionen

<code>def f(x, digits=1):</code>	Default-Wert
<code>return x</code>	ohne return: None
<code>f(3, digits=0)</code>	benanntes Argument
<code>return best, hits</code>	Rückgabe als Paar
<code>best, hits = score_stats(xs)</code>	Auspacken — auch im for
<code>print</code> zeigt nur an, <code>return</code> gibt zurück.	

Listen

Hier steht `xs` für eine Liste, `x` für ein einzelnes Element und `i` für eine Position.

Werkzeug	Tut was	Gibt zurück
<code>len(xs)</code>	Anzahl der Elemente	<code>int</code>
<code>xs[i], xs[-1]</code>	Element an Position i (ab 0), letztes	das Element
<code>xs[1:3]</code>	Slice: Positionen 1 und 2 — Start ein-, Ende ausgeschlossen	neue Liste
<code>x in xs</code>	Enthalten-Prüfung	<code>bool</code>
<code>xs.append(x)</code>	hängt hinten an	None — verändert xs
<code>xs.insert(i, x)</code>	fügt an Position i ein	None — verändert xs
<code>xs.remove(x)</code>	entfernt das erste Vorkommen	None — verändert xs
<code>xs.pop()</code>	entfernt das letzte Element	das Element
<code>xs.sort()</code> / <code>sorted(xs)</code>	sortiert an Ort und Stelle / als Kopie	None / neue Liste
<code>xs.count(x)</code> , <code>xs.index(x)</code>	zählt bzw. findet x	<code>int</code>
<code>max(xs)</code> , <code>min(xs)</code>	größtes, kleinstes Element	das Element
<code>sum(xs)</code>	Summe der Zahlen	eine Zahl

None -Methoden wirken auf die Liste — nie `xs = xs.append(x)`.

Namen & Objekte

Schreibweise	Was passiert
<code>b = a</code>	zweites Etikett, keine Kopie
<code>a.append(3)</code> , <code>a[0] = 7</code>	Objekt verändert — jeder Name sieht es
<code>a = a + [3]</code> , <code>x = x + 1</code>	neues Objekt, Name umgehängt

Kopie: `list(a)` / `a.copy()`, verschachtelt nur die äußere Ebene. `is` = dasselbe Objekt, `==` = gleicher Inhalt. Unveränderlich: `int`, `float`, `str`, `tuple`. Funktionen erhalten die Liste selbst, keine Kopie.